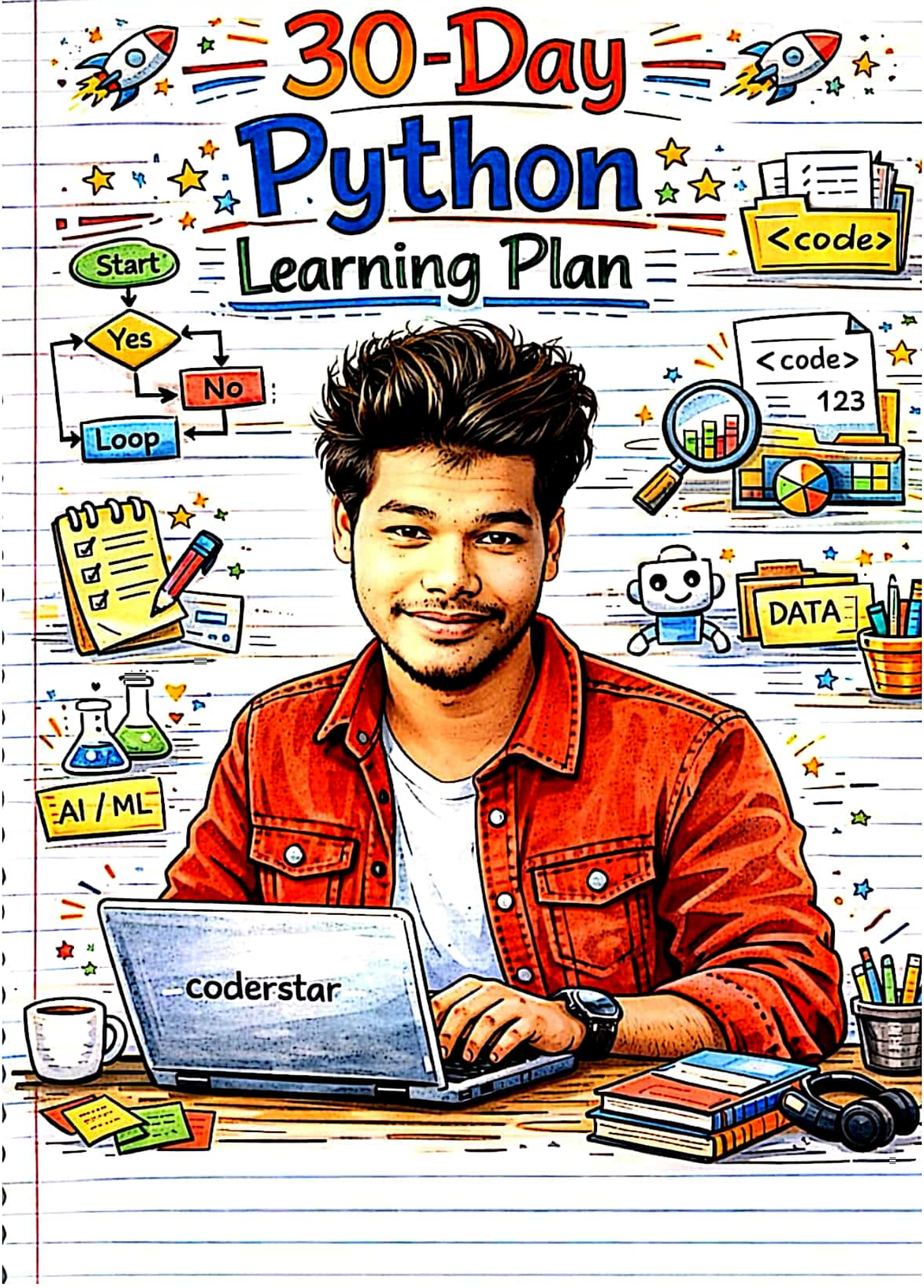


Scanned with CamScanner

Scanned with CamScanner

In [11]:

```
# Factorial of a number using recursion

def recur_factorial(n):
    if n == 1:
        return n
    else:
        return n*recur_factorial(n-1)

num = int(input("Enter the number: "))

# check if the number is negative if
num < 0:

    print("Sorry, factorial does not exist for negative numbers")
elif num == 0:
    print("The factorial of 0 is 1")
else:
    print("The factorial of", num, "is", recur_factorial(num))
```

```
Enter the number: 7
The factorial of 7 is 5040
```

Program 29

Write a Python Program to calculate your Body Mass Index.

Body Mass Index (BMI) is a measure of body fat based on an individual's weight and height. It is commonly used as a screening tool to categorize individuals into different weight status categories, such as underweight, normal weight, overweight, and obesity.

The BMI is calculated using the following formula:

$$\text{BMI} = \frac{\text{Weight (kg)}}{\text{Height (m)}2}$$

Alternatively, in the imperial system:

$$\text{BMI} = \frac{\text{Weight (lb)}}{\text{Height (in)}2} \times 703$$

BMI provides a general indication of body fatness but does not directly measure body fat or distribution. It is widely used in public health and clinical settings as a quick and simple tool to assess potential health risks associated with weight. Different BMI ranges are associated with different health categories, but it's important to note that BMI has limitations and does not account for factors such as muscle mass or distribution of fat.

Scanned with CamScanner

In [12]:

```
def bodymassindex(height, weight):
    return round((weight / height**2),2)


h = float(input("Enter your height in meters: "))
w = float(input("Enter your weight in kg: "))


print("Welcome to the BMI calculator.")

bmi = bodymassindex(h, w) print("Your BMI is: ",
bmi)


if bmi <= 18.5:

    print("You are underweight.")
elif 18.5 < bmi <= 24.9:
    print("Your weight is normal.")
elif 25 < bmi <= 29.29:
    print("You are overweight.")
else:
    print("You are obese.")
```

```
Enter your height in meters: 1.8
Enter your weight in kg: 70
Welcome to the BMI calculator.
Your BMI is: 21.6
Your weight is normal.
```

Program 30

Write a Python Program to calculate the natural logarithm of any number.

The natural logarithm often denoted as ln, is a mathematical function that represents the logarithm to the base e, where is e the mathematical constant approximately equal to 2.71828. In other words, for a positive number x, the natural logarithm of x is the exponent y that satisfies the equation $ey=x$.

Mathematically, the natural logarithm is expressed as:

$$\ln(x)$$

It is commonly used in various branches of mathematics, especially in calculus and mathematical analysis, as well as in fields such as physics, economics, and engineering. The natural logarithm has properties that make it particularly useful in situations involving exponential growth or decay.

Comment 'Python' below, and I'll DM you the complete PDF link 🔗.

Scanned with CamScanner

In [9]:

```
# Python program to display the Fibonacci sequence

def recur_fibo(n):
if n <= 1:
        return n
    else:
        return(recur_fibo(n-1) + recur_fibo(n-2))

nterms = int(input("Enter the number of terms (greater than 0): "))

# check if the number of terms is valid if nterms <= 0:


    print("Plese enter a positive integer")
else:
    print("Fibonacci sequence:")
    for i in range(nterms):
        print(recur_fibo(i))
```

```
Enter the number of terms (greater than 0): 8
Fibonacci sequence: 0 1 1 2 3 5 8 13
```

Program 28

Write a Python Program to Find Factorial of Number Using Recursion.

The factorial of a non-negative integer (n) is the product of all positive integers less than or equal to (n). It is denoted by (n!) and is defined as:

$$n!=n\times(n-1)\times(n-2)\times...\times3\times2\times1$$

For example:

- $5!=5\times4\times3\times2\times1=120$
- $0!$ is defined to be 1.

Factorials are commonly used in mathematics, especially in combinatorics and probability, to count the number of ways a set of elements can be arranged or selected.

Comment 'Python' below, and I'll DM you the complete PDF link 🔗.

Scanned with CamScanner

For two numbers a and b, the HCF can be found using the formula:

$$\text{HCF}(a, b) = \text{GCD}(a, b)$$

For more than two numbers, you can find the HCF by taking the GCD of pairs of numbers at a time until you reach the last pair.

Note: GCD stands for Greatest Common Divisor.

In [2]:
```
# Python program to find H.C.F of two
numbers

# define a function def compute_hcf(x, y):

# choose the smaller number
    if x > y:
        smaller = y
    else:
        smaller = x
    for i in range(1, smaller+1):
        if((x % i == 0) and (y % i == 0)):
            hcf = i
    return hcf

num1 = int(input('Enter the number: ')) num2 =
int(input('Enter the number: '))

print("The H.C.F. is", compute_hcf(num1, num2))
```

```
Enter the number: 54
Enter the number: 24
The H.C.F. is 6
```

Program 24

Write a Python Program to Convert Decimal to Binary, Octal and Hexadecimal.

How can we manually convert a decimal number to binary, octal and hexadecimal?

Converting a decimal number to binary, octal, and hexadecimal involves dividing the decimal number by the base repeatedly and noting the remainders at each step. Here's a simple example:

Let's convert the decimal number 27 to binary, octal, and hexadecimal.

1. Binary:
    - Divide 27 by 2. Quotient is 13, remainder is 1. Note the remainder.
    - Divide 13 by 2. Quotient is 6, remainder is 1. Note the remainder.
    - Divide 6 by 2. Quotient is 3, remainder is 0. Note the remainder.
    - Divide 3 by 2. Quotient is 1, remainder is 1. Note the remainder.
    - Divide 1 by 2. Quotient is 0, remainder is 1. Note the remainder.

    Reading the remainders from bottom to top, the binary representation of 27 is 11011.
2. Octal:

Scanned with CamScanner

In [5]:

```
# This function adds two numbers
def add(x, y):
    return x + y

# This function subtracts two numbers
def subtract(x, y):
    return x - y

# This function multiplies two numbers
def multiply(x, y):
    return x * y

# This function divides two numbers
def divide(x, y):
    return x / y


print("Select  operation.")
print("1.Add")
print("2.Subtract")
print("3.Multiply")
print("4.Divide")

while True:
    # take input from the user
    choice = input("Enter choice(1/2/3/4): ")

    # check if choice is one of the four options
    if choice in ('1', '2', '3', '4'):
        try:
            num1 = float(input("Enter first number: "))
            num2 = float(input("Enter second number: "))
        except ValueError:
            print("Invalid input. Please enter a number.")
            continue

        if choice == '1':
            print(num1, "+", num2, "=", add(num1, num2))

        elif choice == '2':
            print(num1, "-", num2, "=", subtract(num1, num2))

        elif choice == '3':
            print(num1, "*", num2, "=", multiply(num1, num2))

        elif choice == '4':
            print(num1, "/", num2, "=", divide(num1, num2))

        # check if user wants another calculation
        # break the while loop if answer is no
        next_calculation = input("Let's do next calculation? (yes/no):
        if next_calculation == "no":
          break
    else:
        print("Invalid Input")
```

Scanned with CamScanner

- Divide 27 by 8. Quotient is 3, remainder is 3. Note the remainder.
- Divide 3 by 8. Quotient is 0, remainder is 3. Note the remainder.

Reading the remainders from bottom to top, the octal representation of 27 is 33.

3. Hexadecimal:

   - Divide 27 by 16. Quotient is 1, remainder is 11 (B in hexadecimal). Note the remainder.

   Reading the remainders, the hexadecimal representation of 27 is 1B.

So, in summary:

- Binary: 27 in decimal is 11011 in binary.
- Octal: 27 in decimal is 33 in octal.
- Hexadecimal: 27 in decimal is 1B in hexadecimal.

In [3]:
```
dec_num = int(input('Enter a decimal
number: '))

print("The decimal value of", dec_num,
"is:") print(bin(dec_num), "in binary.")
print(oct(dec_num), "in octal.")
print(hex(dec_num), "in hexadecimal.")
```

```
Enter a decimal number: 27
The decimal value of 27 is:
0b11011 in binary.
0o33 in octal.
0x1b in hexadecimal.
```

Program 25

Write a Python Program To Find ASCII value of a character.

ASCII value:

ASCII, or American Standard Code for Information Interchange, is a character encoding standard that uses numeric values to represent characters. Each ASCII character is assigned a unique 7-bit or 8-bit binary number, allowing computers to exchange information and display text in a consistent way. The ASCII values range from 0 to 127 (for 7-bit ASCII) or 0 to 255 (for 8-bit ASCII), with each value corresponding to a specific character, such as letters, digits, punctuation marks, and control characters.

In [4]:
```
char = str(input("Enter the character: "))
print("The ASCII value of '" + char + "' is", ord(char))
```

```
Enter the character: P
The ASCII value of 'P' is 80
```

Program 26

Write a Python Program to Make a Simple Calculator with 4 basic mathematical operations.

Scanned with CamScanner

In [26]:

```
# Input the interval from the user lower = int(input("Enter
the lower limit of the interval: ")) upper = int(input("Enter
the upper limit of the interval: "))


for num in range(lower, upper + 1): # Iterate through the numbers i
    order = len(str(num)) # Find the number of digits in 'num'
    temp_num = num
    sum = 0

    while temp_num > 0:
        digit = temp_num % 10
        sum += digit ** order
        temp_num //= 10

    # Check if 'num' is an Armstrong number
    if num == sum: print(num)
```

```
Enter the lower limit of the interval: 10
Enter the upper limit of the interval: 1000
153 370 371 407
```

Program 21

Write a Python Program to Find the Sum of Natural Numbers.

Natural numbers are a set of positive integers that are used to count and order objects. They are the numbers that typically start from 1 and continue indefinitely, including all the whole numbers greater than 0. In mathematical notation, the set of natural numbers is often denoted as "N" and can be expressed as:

$$N = 1,2,3,4,5,6,7,8,...$$

In [27]:

```
limit = int(input("Enter the limit: "))

# Initialize the sum sum = 0

# Use a for loop to calculate the sum
of natural numbers for i in range(1,
limit + 1):
    sum += i

# Print the sum print("The sum of natural
numbers up to", limit, "is:", sum)
```

```
Enter the limit: 10
The sum of natural numbers up to 10 is: 55
```

Scanned with CamScanner

Program 22

Write a Python Program to Find LCM.

Least Common Multiple (LCM):

LCM, or Least Common Multiple, is the smallest multiple that is exactly divisible by two or more numbers.

Formula:

For two numbers a and b, the LCM can be found using the formula:

$$\text{LCM}(a, b) = \frac{|a \cdot b|}{\text{GCD}(a, b)}$$

For more than two numbers, you can find the LCM step by step, taking the LCM of pairs of numbers at a time until you reach the last pair.

Note: GCD stands for Greatest Common Divisor.

In [1]:
```
# Python Program to find the L.C.M. of two input number

def compute_lcm(x, y):

    if x > y:                    # choose the greater number
        greater = x
    else:
        greater = y
    while(True):
        if((greater % x == 0) and (greater % y == 0)):
            lcm = greater
            break
        greater += 1
    return lcm

num1 = int(input('Enter the number: ')) num2 =
int(input('Enter the number: '))

print("The L.C.M. is", compute_lcm(num1, num2))
```

```
Enter the number: 54
Enter the number: 24
The L.C.M. is 216
```

Program 23

Write a Python Program to Find HCF.

Highest Common Factor(HCF):

HCF, or Highest Common Factor, is the largest positive integer that divides two or more numbers without leaving a remainder.

Formula:

Scanned with CamScanner

- If we calculate $9^4+4^4+4^4+7^4$, we get $6561 + 256 + 2401 + 256$, which is also equal to 9474.

Therefore, 9474 is an Armstrong number as well.

In [25]:

```
num = int(input("Enter a number: "))

# Calculate the number of digits in num num_str =
str(num) num_digits = len(num_str)

# Initialize variables sum_of_powers = 0 temp_num =
num

# Calculate the sum of digits raised to the power of
num_digits

while temp_num > 0:


    digit = temp_num % 10
    sum_of_powers += digit ** num_digits
    temp_num //= 10

# Check if it's an Armstrong number
if sum_of_powers == num:
    print(f"{num} is an Armstrong number.")
else:
    print(f"{num} is not an Armstrong number.")

```

```
Enter a number: 9474
9474 is an Armstrong number.
```

Program 20

Write a Python Program to Find Armstrong Number in an Interval.

Comment 'Python' below, and I'll DM you the complete PDF link 🔗.

Scanned with CamScanner

Program 20

Write a Python Program to Find Armstrong Number in an Interval.

In [26]:

```
# Input the interval from the user lower = int(input("Enter
the lower limit of the interval: ")) upper = int(input("Enter
the upper limit of the interval: "))


for num in range(lower, upper + 1): # Iterate through the numbers i
    order = len(str(num))  # Find the number of digits in 'num'
    temp_num = num
    sum = 0

    while temp_num > 0:
        digit = temp_num % 10
        sum += digit ** order
        temp_num //= 10

    # Check if 'num' is an Armstrong number
    if num == sum: print(num)
```

```
Enter the lower limit of the interval: 10
Enter the upper limit of the interval: 1000
153 370 371 407
```

Program 21

Write a Python Program to Find the Sum of Natural Numbers.

Natural numbers are a set of positive integers that are used to count and order objects. They are the numbers that typically start from 1 and continue indefinitely, including all the whole numbers greater than 0. In mathematical notation, the set of natural numbers is often denoted as "N" and can be expressed as:

$$N = 1,2,3,4,5,6,7,8,\ldots$$

In [27]:

```
limit = int(input("Enter the limit: "))

# Initialize the sum sum = 0

# Use a for loop to calculate the sum of natural numbers
for i in range(1, limit + 1):

    sum += i

# Print the sum print("The sum of natural numbers up to",
limit, "is:", sum)
```

```
Enter the limit: 10
The sum of natural numbers up to 10 is: 55
```


Scanned with CamScanner

Program 19

Write a Python Program to Check Armstrong Number?

Armstrong Number:

It is a number that is equal to the sum of its own digits, each raised to a power equal to the number of digits in the number.

For example, let's consider the number 153:

- It has three digits (1, 5, and 3).
- If we calculate 13 + 53+33, we get 1+125+27, which is equal to 153.

So, 153 is an Armstrong number because it equals the sum of its digits raised to the power of the number of digits in the number.
Another example is 9474:

- It has four digits (9, 4, 7, and 4).
    - If we calculate $94+44+7^4+4^4$, we get 6561 + 256 + 2401 + 256, which is also equal to 9474.

Therefore, 9474 is an Armstrong number as well.

In [25]:

```
num = int(input("Enter a number: "))

# Calculate the number of digits in num num_str = str(num)
num_digits = len(num_str)

# Initialize variables sum_of_powers = 0 temp_num = num

# Calculate the sum of digits raised to the power of num_digits

while temp_num > 0:




    digit = temp_num % 10
    sum_of_powers += digit ** num_digits
    temp_num //= 10

# Check if it's an Armstrong number
if sum_of_powers == num:
    print(f"{num} is an Armstrong number.")
else:
    print(f"{num} is not an Armstrong number.")

```

```
Enter a number: 9474
9474 is an Armstrong number.
```


Scanned with CamScanner

In [22]:
```
num = int(input("Display multiplication table of: "))

for i in range(1, 11):
    print(f"{num} X {i} = {num*i}")
```

```
Display multiplication table of: 19
19 X 1 = 19 19 X 2 = 38 19 X 3 = 57
19 X 4 = 76 19 X 5 = 95 19 X 6 =
114 19 X 7 = 133 19 X 8 = 152 19 X
9 = 171 19 X 10 = 190
```

Program 18

Write a Python Program to Print the Fibonacci sequence.

Fibonacci sequence:

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, typically starting with 0 and 1. So, the sequence begins with 0 and 1, and the next number is obtained by adding the previous two numbers. This pattern continues indefinitely, generating a sequence that looks like this:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, and so on.

Mathematically, the Fibonacci sequence can be defined using the following recurrence relation:

$F(0)=0\ F(1)=1\ F(n)=F(n-1)+F(n-2) for n>1$

In [23]:
```
nterms = int(input("How many terms? "))

# first two terms n1, n2 = 0, 1 count = 0

# check if the number of terms is valid if nterms
<= 0:



    print("Please enter a positive integer")
# if there is only one term, return n1
elif nterms == 1:
    print("Fibonacci sequence upto",nterms,":")
    print(n1)
# generate fibonacci sequence
else:
    print("Fibonacci sequence:")
    while count < nterms:
       print(n1)
       nth = n1 + n2
       # update values
        n1 = n2
        n2 = nth
        count += 1
```

```
How many terms? 10 Fibonacci
sequence: 0 1 1 2 3 5 8 13 21 34
```


Scanned with CamScanner

Program 14

Write a Python Program to Check Prime Number.

Prime Numbers:

A prime number is a whole number that cannot be evenly divided by any other number except for 1 and itself. For example, 2, 3, 5, 7, 11, and 13 are prime numbers because they cannot be divided by any other positive integer except for 1 and their own value.

In [15]:

```
num = int(input("Enter a number: "))

# define a flag variable
flag = False

if num == 1:
    print(f"{num}, is not a prime number")
elif num > 1:
    # check for factors
    for i in range(2, num):
        if (num % i) == 0:
            flag = True     # if factor is found, set flag to True
            # break out of loop
            break

    # check if flag is True
if flag:
    print(f"{num}, is not a prime number")
else:
    print(f"{num}, is a prime number")
```

```
Enter a number: 27
27, is not a prime number
```

Program 15

Write a Python Program to Print all Prime Numbers in an Interval of 1-10.

Comment 'Python' below, and I'll DM you the complete PDF link 🔗.

Scanned with CamScanner

In [20]:

```
# Python program to display all the prime numbers within an interval

lower = 1 upper = 10

print("Prime numbers between", lower, "and", upper, "are:")

for num in range(lower, upper + 1):

   # all prime numbers are greater than 1
   if num > 1:
        for i in range(2, num):
            if (num % i) == 0:
                break
        else:
            print(num)
```

```
Prime numbers between 1 and 10 are:
2 3 5 7
```

Program 16

Write a Python Program to Find the Factorial of a Number.

In [21]:

```
num = int(input("Enter a number: "))
factorial = 1
if num <0:
    print("Factirial does not exist for negative numbers")
elif num == 0:
    print("Factorial of 0 is 1")
else:
    for i in range(1, num+1):
        factorial = factorial*i
    print(f'The factorial of {num} is {factorial}')
```

```
Enter a number: 4
The factorial of 4 is 24
```

Program 17

Write a Python Program to Display the multiplication Table.

Scanned with CamScanner

Program 11

Write a Python Program to Check if a Number is Positive, Negative or Zero.

In [12]:
```
num = float(input("Enter a number: "))
if num > 0:
   print("Positive number")
elif num == 0:
   print("Zero")
else:
   print("Negative number")
```

```
Enter a number: 6.4
Positive number
```

Program 12

Write a Python Program to Check if a Number is Odd or Even.

In [13]:
```
num = int(input("Enter a number: "))

i f   n u m % 2   = =   0 :
    print("This is a even number")
else:
    print("This is a odd number")
```

```
Enter a number: 3
This is a odd number
```

Program 13

Write a Python Program to Check Leap Year.

In [14]:
```
year = int(input("Enter a year: "))

# divided by 100 means century year (ending with 00) # century year
divided by 400 is leap year if (year % 400 == 0) and (year % 100 == 0):

    print("{0} is a leap year".format(year))

# not divided by 100 means not a century year # year divided by 4
is a leap year elif (year % 4 ==0) and (year % 100 != 0):

    print("{0} is a leap year".format(year))

# if not divided by both 400 (century year) and 4 (not century year)
# year is not leap year
else:
    print("{0} is not a leap year".format(year))
```

```
Enter a year: 2024
2024 is a leap year
```


Scanned with CamScanner

In [8]:
```
celsius = float(input("Enter temperature in Celsius: "))

# Conversion formula: Fahrenheit = (Celsius * 9/5) + 32 fahrenheit =
(celsius * 9/5) + 32

print(f"{celsius} degrees Celsius is equal to {fahrenheit} degrees Fahr
```

```
Enter temperature in Celsius: 37
37.0 degrees Celsius is equal to 98.6 degrees Fahrenheit
```

Program 8

Write a Python program to display calendar.

In [9]:
```
import calendar

year = int(input("Enter year: ")) month = int(input("Enter month: "))

cal = calendar.month(year, month)

print(cal)
```

```
Enter year: 2023
Enter month: 11
   November 2023
Mo Tu We Th Fr Sa Su
            1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30
```

Program 9

Write a Python program to solve quadratic equation.

The standard form of a quadratic equation is:

$$ax2 + bx + c = 0$$

where

a, b and c are real numbers and

$a \neq 0$

The solutions of this quadratic equation is given by:

$$(-b \pm (b^2 - 4ac)1/2)/(2a)$$

Scanned with CamScanner

In [10]:

```
import math

# Input coefficients a = float(input("Enter coefficient a: ")) b =
float(input("Enter coefficient b: ")) c = float(input("Enter
coefficient c: "))

# Calculate the discriminant discriminant = b**2 - 4*a*c

# Check if the discriminant is positive, negative, or zero if
discriminant > 0:

# Two real and distinct roots root1 = (-b +
math.sqrt(discriminant)) / (2*a) root2 = (-b -
math.sqrt(discriminant)) / (2*a) print(f"Root 1: {root1}")
print(f"Root 2: {root2}")


elif discriminant == 0:
    # One real root (repeated)
    root = -b / (2*a)
    print(f"Root: {root}")
else:
    # Complex roots real_part = -b / (2*a) imaginary_part
    = math.sqrt(abs(discriminant)) / (2*a) print(f"Root 1:
    {real_part} + {imaginary_part}i") print(f"Root 2:
    {real_part} - {imaginary_part}i")


```

```
Enter coefficient a: 1
Enter coefficient b: 4
Enter coefficient c: 8
Root 1: -2.0 + 2.0i
Root 2: -2.0 - 2.0i
```

Program 10

Write a Python program to swap two variables without temp variable.

In [11]:

```
a = 5 b = 10

# Swapping without a temporary variable
a, b = b, a


print("After swapping:")
print("a =", a)
print("b =", b)


```

```
After swapping:
a = 10
b = 5
```


Scanned with CamScanner

Program 4

Write a Python program to swap two variables.

In [5]:
```python
# Input two variables
a = input("Enter the value of the first variable (a): ")
b = input("Enter the value of the second variable (b): ")
# Display the original values
print(f"Original values: a = {a}, b = {b}")
# Swap the values using a temporary variable
temp = a
a = b
b = temp
# Display the swapped values
print(f"Swapped values: a = {a}, b = {b}")
```

```
Enter the value of the first variable (a): 5
Enter the value of the second variable (b): 9
Original values: a = 5, b = 9
Swapped values: a = 9, b = 5
```

Program 5

Write a Python program to generate a random number.

In [6]:
```python
import random
print(f"Random number: {random.randint(1, 100)}")
```

```
Random number: 89
```

Program 6

Write a Python program to convert kilometers to miles.

In [7]:
```python
kilometers = float(input("Enter distance in kilometers: "))

# Conversion factor: 1 kilometer = 0.621371 miles
 conversion_factor = 0.621371

miles = kilometers * conversion_factor


print(f"{kilometers} kilometers is equal to {miles} miles")
```

```
Enter distance in kilometers: 100
100.0 kilometers is equal to 62.137100000000004 miles
```

Program 7

Write a Python program to convert Celsius to Fahrenheit.

Comment 'Python' below, and I'll DM you the complete PDF link 🔗.

Scanned with CamScanner

140+ BASIC TO ADVANCED PYTHON PROGRAMS

Program 1

Write a Python program to print "Hello Python".

In [1]:
```
1 print("Hello Python")
```

```
Hello Python
```

Program 2

Write a Python program to do arithmetical operations addition and division.

In [2]:
```
1 # Addition
2 num1 = float(input("Enter the first number for addition: "))
3 num2 = float(input("Enter the second number for addition: "))
4 sum_result = num1 + num2
5 print(f"sum: {num1} + {num2} = {sum_result}")
```

```
Enter the first number for addition: 5
Enter the second number for addition: 6
sum: 5.0 + 6.0 = 11.0
```

In [3]:
```
1 # Division
2 num3 = float(input("Enter the dividend for division: "))
3 num4 = float(input("Enter the divisor for division: "))
4 if num4 == 0:
5     print("Error: Division by zero is not allowed.")
6 else:
7     div_result = num3 / num4
8     print(f"Division: {num3} / {num4} = {div_result}")
```

```
Enter the dividend for division: 25
Enter the divisor for division: 5
Division: 25.0 / 5.0 = 5.0
```

Program 3

Write a Python program to find the area of a triangle.

In [4]:
```
1 # Input the base and height from the user
2 base = float(input("Enter the length of the base of the triangle: "))
3 height = float(input("Enter the height of the triangle: "))
4 # Calculate the area of the triangle
5 area = 0.5 * base * height
6 # Display the result
7 print(f"The area of the triangle is: {area}")
```

```
Enter the length of the base of the triangle: 10
Enter the height of the triangle: 15
The area of the triangle is: 75.0
```

Scanned with CamScanner